

BANKING DASHBOARD & RISK ANALYTICS REPORT

Abstract

The Banking Dashboard & Risk Analytics project focuses on analyzing banking customer data to understand financial behavior, deposits, account usage, and risk-related insights. The project combines SQL, Python, Exploratory Data Analysis (EDA), and Power BI to transform raw banking data into meaningful business insights and interactive dashboards.

This project helps financial institutions understand customer patterns, monitor account activities, identify high-value customers, and reduce financial risks while lending to customers. The dashboard provides data-driven decision-making support using visual analytics and business intelligence techniques.

Problem Statement

Develop a basic understanding of risk analytics in banking and financial services and understand how data is used to minimise the risk of losing money while lending to customers.

Introduction

In the modern banking sector, huge volumes of customer and transaction data are generated every day. Banks use this data to understand customer behavior, monitor account activities, improve customer services, and reduce financial risk.

This project demonstrates how banking data can be analyzed using data analytics tools and visualization techniques. The project uses Python for Exploratory Data Analysis (EDA), SQL for data extraction and management, and Power BI for dashboard creation and business reporting.

The final dashboard provides a clear visual representation of banking performance, customer account distribution, deposits, and financial patterns.

Objectives

- To analyze banking customer data using data analytics techniques.
 - To understand customer account behavior and banking trends.
 - To perform Exploratory Data Analysis (EDA).
 - To create an interactive dashboard using Power BI.
 - To identify insights useful for risk analytics in banking.
 - To improve decision-making using visual reports and business intelligence.
-

Technologies Used

Technology	Purpose
Python	Data analysis and EDA
Pandas	Data manipulation
NumPy	Numerical operations
Matplotlib	Data visualization
Seaborn	Statistical visualization
SQL	Database querying
PostgreSQL	Database management
Power BI	Dashboard and reporting
Jupyter Notebook	Data analysis environment

Project Architecture

1. Data Collection
 2. Data Cleaning
 3. SQL Data Extraction
 4. Exploratory Data Analysis (EDA)
 5. Visualization & Insights
 6. Dashboard Development in Power BI
 7. Business Insights & Reporting
-

Dataset Description

The project uses banking customer data containing information related to:

- Customer Accounts
- Savings Accounts

- Checking Accounts
- Bank Deposits
- Foreign Currency Accounts
- Customer Financial Activities
- Banking Transactions

The dataset is analyzed to identify patterns and relationships among different banking attributes.

SQL Operations Used

The project also involved SQL-based data extraction and management.

Operations Performed

- Database Connection
- Data Retrieval
- Table Queries
- Data Filtering
- Data Analysis Using SQL

Example SQL Query:

```
SELECT * FROM customer;
```

Dataset Preparation & Database Connection

SQL Data Import

Query

Query History

1

select * from customer;

2

Data Output

Messages

Notifications

The banking dataset was imported into PostgreSQL for structured storage and database management. The database setup helped organize the data efficiently before performing analysis.

Database Connection with Jupyter Notebook

Database Connection

```
[32]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

[33]: import psycopg2
# Connect to server
conn = psycopg2.connect(
    dbname="Banking_Case", # your database name
    user="postgres",      # your username
    password="Asim$282003", # your password
    host="localhost",     # same as pgAdmin connection
    port=5432             # default PostgreSQL port
)
```

The PostgreSQL database was successfully connected with Jupyter Notebook using Python libraries. This connection enabled seamless data extraction and analysis directly within the Python environment.

Tools & Libraries Used in Python

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

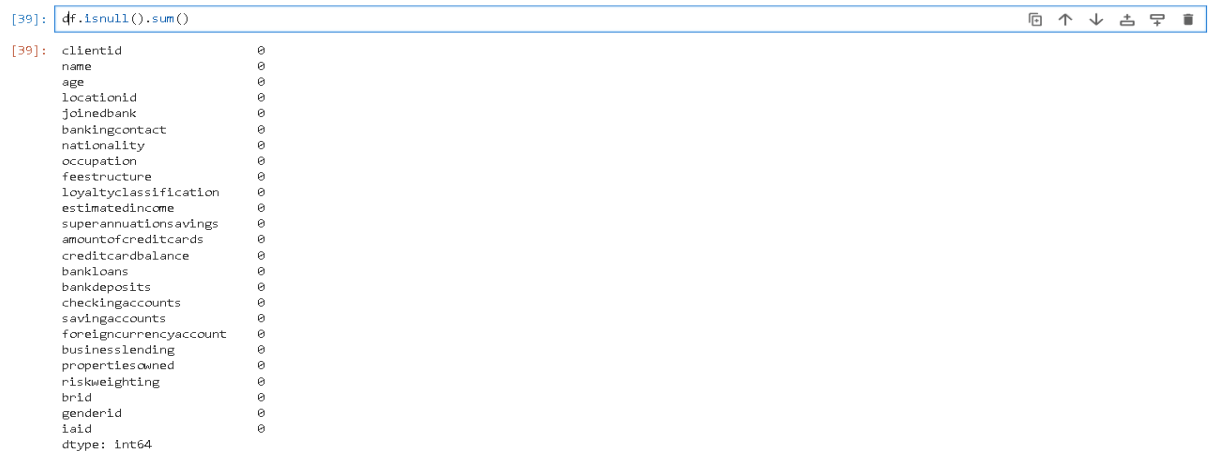
Exploratory Data Analysis (EDA)

The EDA process was performed using Python libraries such as Pandas, Matplotlib, and Seaborn.

Types of Analysis Performed

Data Cleaning & Null Value Analysis

Null Value Analysis



```
[39]: df.isnull().sum()
[39]: clientid      0
      name         0
      age          0
      locationid   0
      joinedbank   0
      bankingcontact 0
      nationality   0
      occupation    0
      feestructure  0
      loyaltyclassification 0
      estimatedincome 0
      superannuationsavings 0
      amountofcreditcards 0
      creditcardbalance 0
      bankloans     0
      bankdeposits  0
      checkingaccounts 0
      savingaccounts 0
      foreigncurrencyaccount 0
      businesslending 0
      propertiesowned 0
      riskweighting  0
      brid          0
      genderid      0
      iid           0
      dtype: int64
```

The dataset was checked for missing values and inconsistencies to ensure accurate analysis and reliable visualizations.

Descriptive Statistics

Descriptive Statistics

```
[40]: # Generate descriptive statistics for the dataframe
df.describe()
```

```
[40]:
```

	age	estimatedincome	superannuationsavings	amountofcreditcards	creditcardbalance	bankloans	bankdeposits	checkingaccounts	savingaccounts
count	3000.000000	3000.000000	3000.000000	3000.000000	3000.000000	3.000000e+03	3.000000e+03	3.000000e+03	3.000000e+03
mean	51.039667	171305.034263	25531.599673	1.463667	3176.206943	5.913862e+05	6.715602e+05	3.210929e+05	2.329084e+05
std	19.854760	111935.808209	16259.950770	0.676387	2497.094709	4.575570e+05	6.457169e+05	2.820796e+05	2.300078e+05
min	17.000000	15919.480000	1482.030000	1.000000	1.170000	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
25%	34.000000	82906.595000	12513.775000	1.000000	1236.630000	2.396281e+05	2.044004e+05	1.199475e+05	7.479440e+04
50%	51.000000	142313.480000	22357.355000	1.000000	2560.805000	4.797934e+05	4.633165e+05	2.428157e+05	1.640866e+05
75%	69.000000	242290.305000	35464.740000	2.000000	4522.632500	8.258130e+05	9.427546e+05	4.348749e+05	3.155750e+05
max	85.000000	522330.260000	75963.900000	3.000000	13991.990000	2.667557e+06	3.890598e+06	1.969923e+06	1.724118e+06

Statistical summaries were generated to understand the distribution, mean, median, and spread of banking attributes.

Unique Categories Analysis

Unique Categories

```
[45]: # Examine the distribution of unique categories in categorical columns

categorical_cols=df[['nationality', 'occupation', 'fee_structure', 'loyalty_classification',
                    'amount_of_credit_cards', 'properties_owned', 'risk_weighting',
                    'br_id', 'gender_id', 'ia_id', 'income_band']].columns

for col in categorical_cols:
    print(f"Value Counts for '{col}':")
    display(df[col].value_counts())
```

```
Value Counts for 'nationality':
nationality
European      1309
Asian         754
American      507
Australian    254
African       176
Name: count, dtype: int64

Value Counts for 'occupation':
occupation
Structural Analysis Engineer    28
Associate Professor             28
Recruiter                      25
Human Resources Manager        24
Account Coordinator            24
..
Office Assistant IV             8
```

Category-wise analysis helped identify different customer groups and banking classifications.

Income Band Analysis

Income Band Analysis

```
[44]: # Categorize continuous income values into Low, Med, High groups

# Define income ranges (bins) :- 0-100000 → Low, 100000-300000 → Med, 300000+ → High

bins = [0,100000,300000,float('inf')]
labels = ['Low','Med','High']

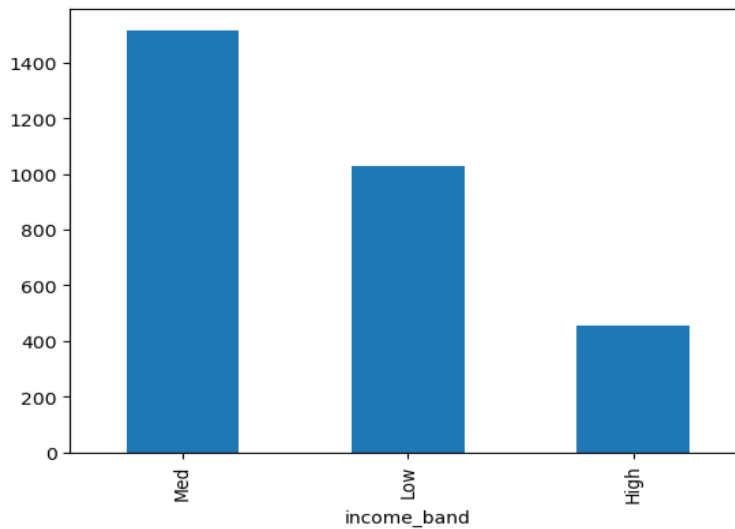
# Categorize 'estimated_income' into bands using pd.cut
# right=False → upper bound excluded (e.g. 100000 goes to 'Med')

df['income_band'] = pd.cut(df['estimated_income'], bins=bins, labels=labels, right=False)

# Plot a bar chart showing how many customers fall into each income band

df['income_band'].value_counts().plot(kind='bar')
```

```
[44]: <Axes: xlabel='income_band'>
```



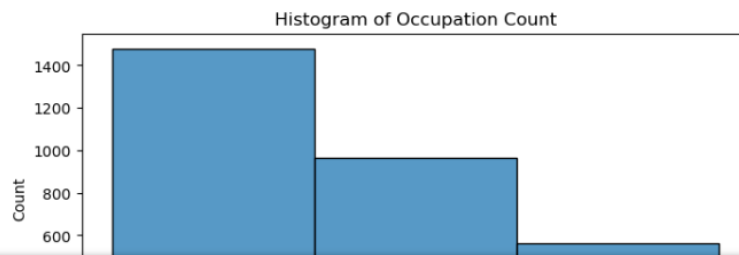
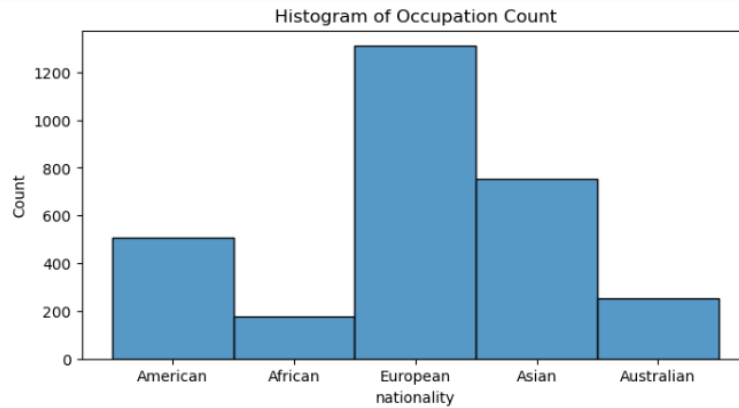
Income segmentation analysis provides insights into customer income groups and their banking activities.

Occupation-based Histplot

Occupation Histplot

```
[48]: # Hisplot of value counts for different occupation
```

```
for col in categorical_cols:
    if col == 'occupation':
        continue
    plt.figure(figsize=(8,4))
    sns.histplot(df[col])
    plt.title('Histogram of Occupation Count')
    plt.xlabel(col)
    plt.ylabel('Count')
    plt.show()
```



Histogram visualizations were used to analyze customer occupation distributions and related banking trends.

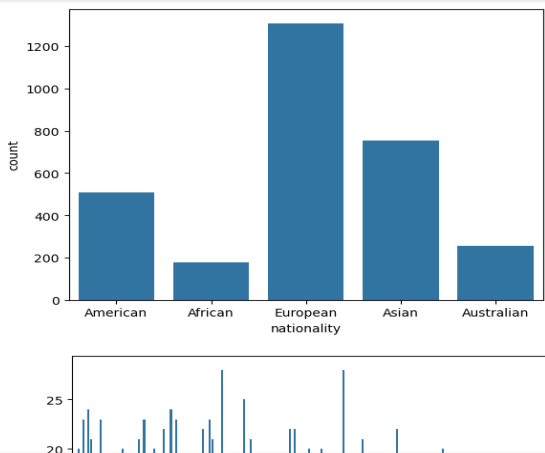
Used to analyze individual variables and understand data distributions.

1. Univariate Analysis

Univariate Analysis

Univariate Analysis

```
[46]: for i, predictor in enumerate(df[['nationality', 'occupation', 'fee_structure',  
    'loyalty_classification', 'amount_of_credit_cards',  
    'properties_owned', 'risk_weighting', 'br_id', 'gender_id',  
    'ia_id', 'income_band']].columns):  
  
    plt.figure(i)  
    sns.countplot(data=df, x=predictor)
```



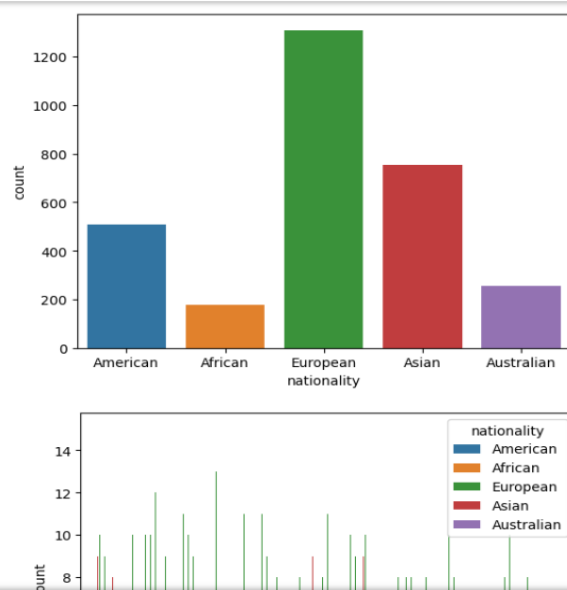
Univariate analysis was performed to study individual variables and understand customer data distributions.

2. Bivariate Analysis

Bivariate Analysis

Bivariate Analysis

```
[47]: for i, predictor in enumerate(df[['nationality', 'occupation', 'fee_structure',  
    'loyalty_classification', 'amount_of_credit_cards',  
    'properties_owned', 'risk_weighting', 'br_id', 'gender_id',  
    'ia_id', 'income_band']].columns):  
  
    plt.figure(i)  
    sns.countplot(data=df, x=predictor, hue='nationality')
```



Bivariate analysis helped identify relationships between different banking variables and customer attributes.

Used to identify relationships between two variables.

3. Numerical Analysis

Numeric Analysis

Numerical Analysis

```
[50]: import math

numerical_cols = ['estimated_income', 'superannuation_savings',
                  'credit_card_balance', 'bank_loans', 'bank_deposits',
                  'checking_accounts', 'saving_accounts',
                  'foreign_currency_account', 'business_lending']

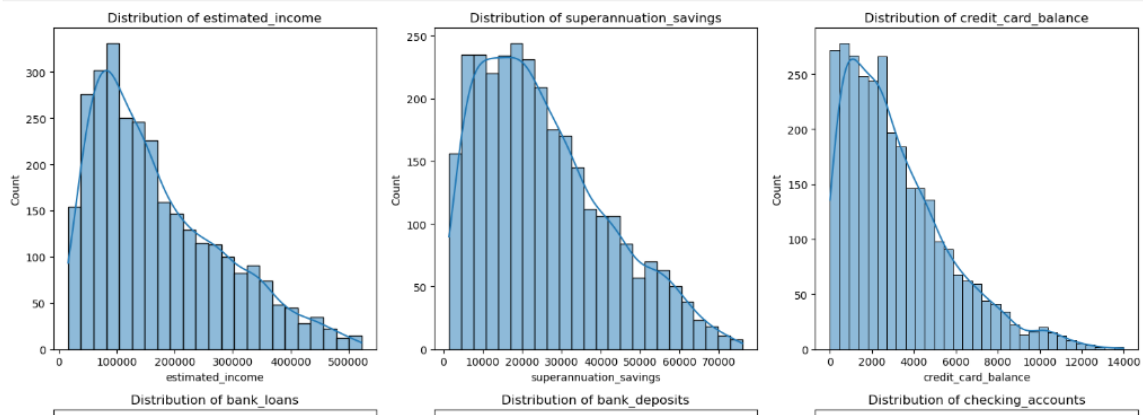
# Univariate analysis and visualization

# Number of plots
n = len(numerical_cols)
rows = math.ceil(n/3) # 3 plots per row + 4 rows for 10 columns

plt.figure(figsize=(15, 5*rows)) # adjust figure size

for i, col in enumerate(numerical_cols, 1):
    plt.subplot(rows, 3, i) # rows x 3 grid
    sns.histplot(df[col], kde=True)
    plt.title(f"Distribution of {col}")

plt.tight_layout()
plt.show()
```



Numerical analysis was performed to evaluate financial metrics and statistical relationships within the dataset.

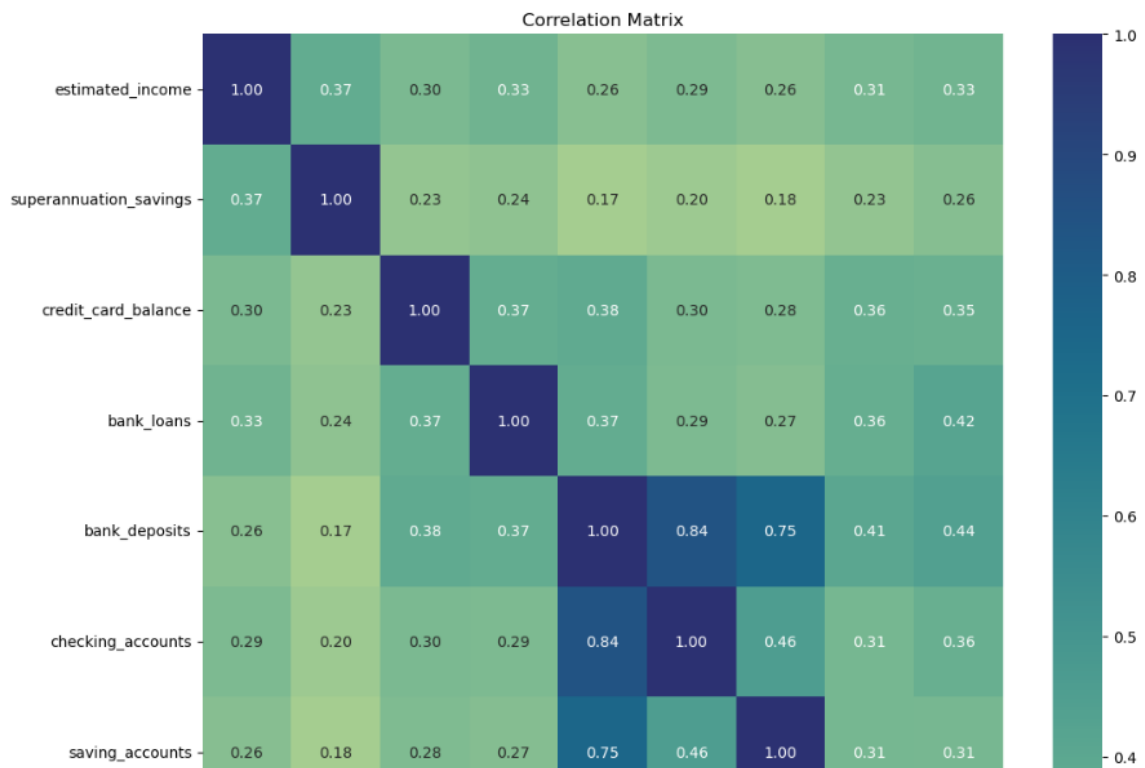
Performed statistical analysis on numerical data.

4. Correlation Matrix

Correlation Matrix

Heatmaps

```
[51]: numerical_cols = ['estimated_income', 'superannuation_savings',  
                    'credit_card_balance', 'bank_loans', 'bank_deposits',  
                    'checking_accounts', 'saving_accounts',  
                    'foreign_currency_account', 'business_lending']  
  
correlation_matrix = df[numerical_cols].corr()  
  
plt.figure(figsize=(12,12))  
sns.heatmap(correlation_matrix, annot=True, cmap='crest', fmt=".2f")  
plt.title("Correlation Matrix")  
plt.show()
```



Correlation matrix were used to identify strong positive and negative relationships between banking attributes.

Key Insights

Insight 1

Strong positive correlations were observed between Bank Deposits, Checking Accounts, Savings Accounts, and Foreign Currency Accounts.

Insight 2

Customers maintaining higher balances in one account type are likely to maintain high balances in other account categories as well.

Insight 3

Data visualization helped identify patterns useful for banking risk analysis and customer segmentation.

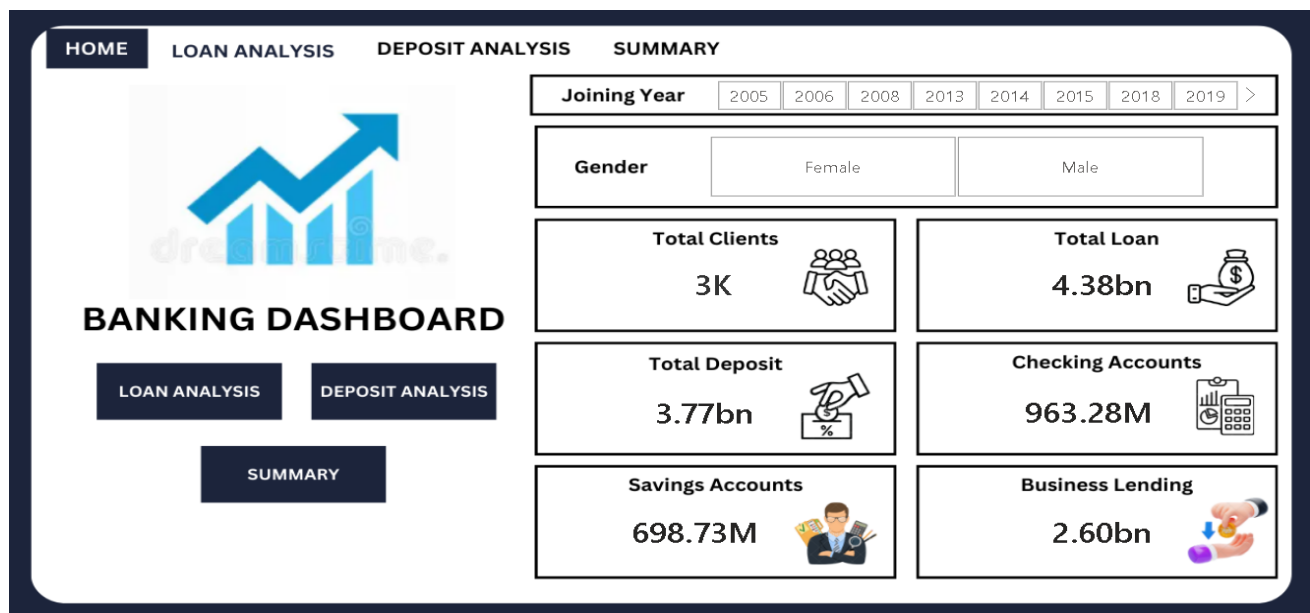
Insight 4

Interactive dashboards improve business decision-making and monitoring efficiency.

Power BI Dashboard

Home Dashboard

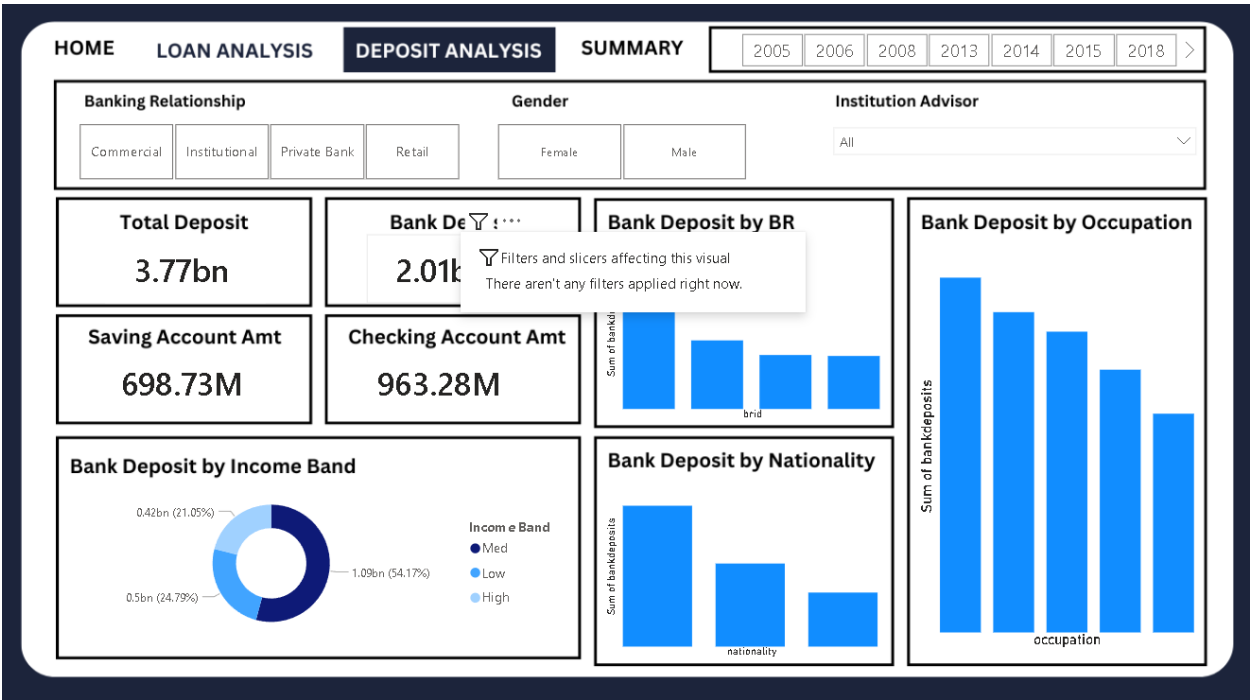
Power BI Home Dashboard



The main Power BI dashboard provides a complete overview of banking performance using KPIs, charts, filters, and interactive visualizations.

Deposit Analysis Dashboard

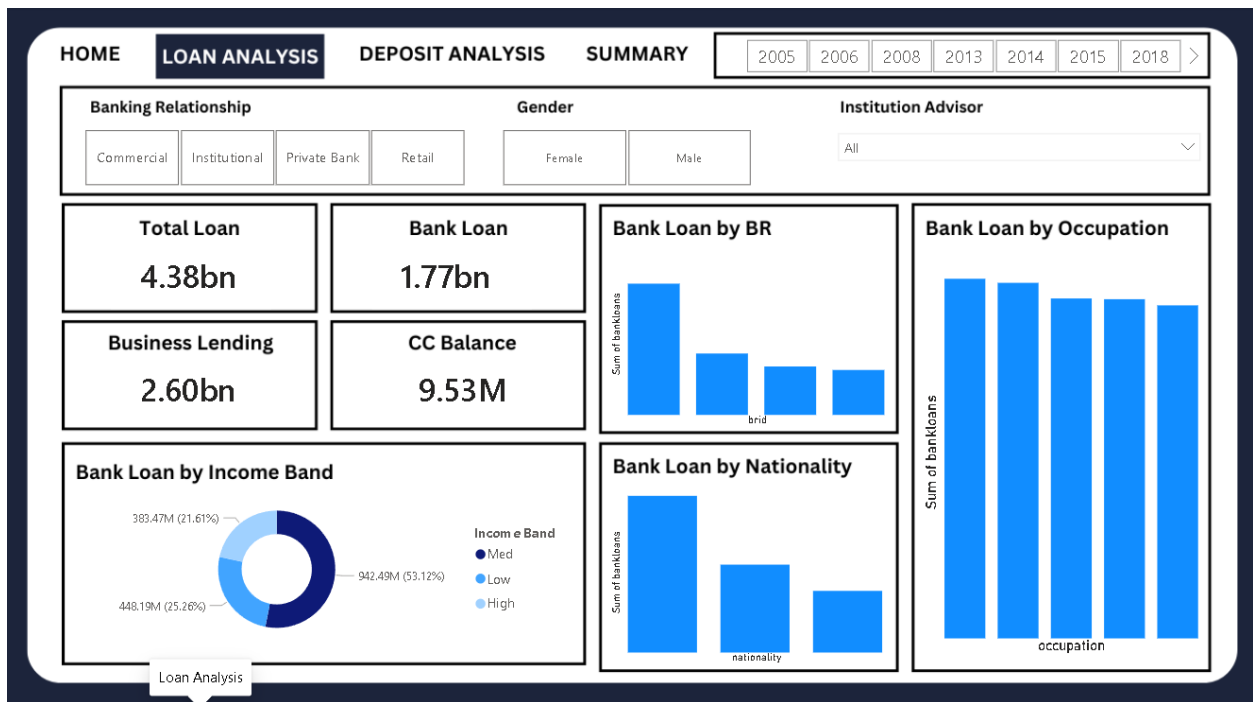
Deposit Analysis



The deposit analysis section helps monitor customer deposit behavior and account balance trends.

Loan Analysis Dashboard

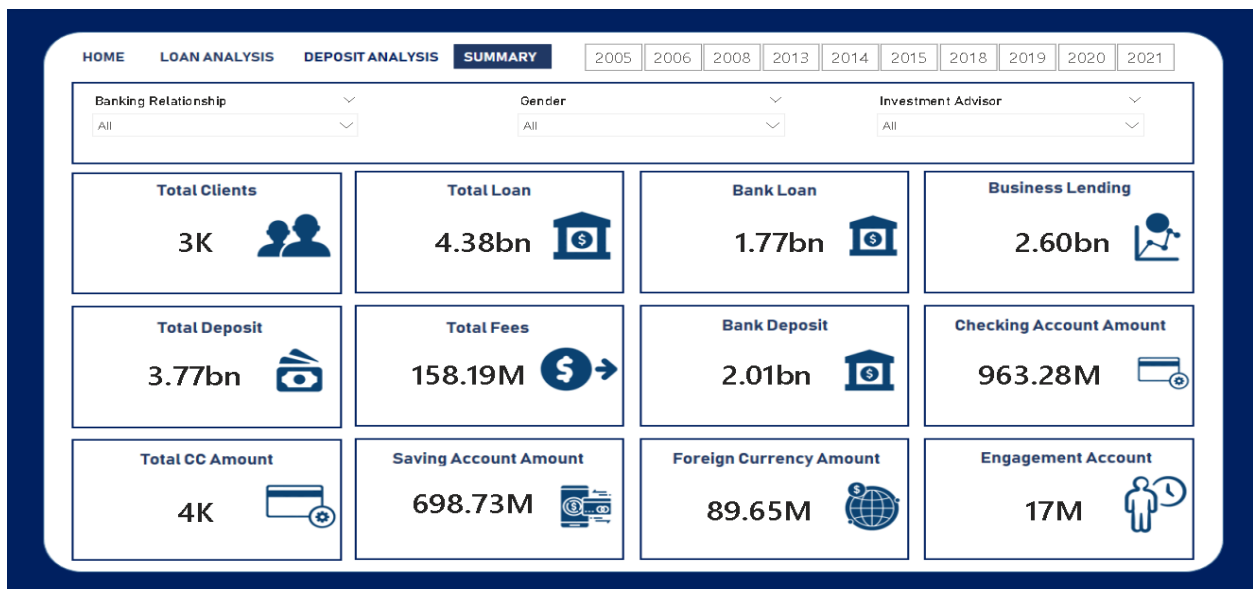
Loan Analysis



Loan analysis visualizations help understand customer lending behavior and support risk analytics.

Summary Analysis

Summary Analysis



Summary reports provided quick insights into overall banking data patterns and customer information.

Dashboard Features

- Interactive Filters
- Banking KPI Monitoring
- Customer Financial Insights
- Deposit Analysis
- Risk Analytics Visualization
- Data-driven Reports

Dashboard Benefits

- Faster Decision Making
 - Better Customer Understanding
 - Improved Financial Monitoring
 - Enhanced Risk Management
-

System Workflow

1. Import Banking Dataset
 2. Connect Database Using SQL
 3. Clean and Process Data
 4. Perform Exploratory Data Analysis
 5. Generate Statistical Insights
 6. Build Interactive Dashboard in Power BI
 7. Present Business Insights
-

Advantages of the Project

- Helps understand customer financial behavior.
 - Supports banking risk analysis.
 - Converts raw data into visual insights.
 - Improves reporting efficiency.
 - Helps management take data-driven decisions.
-

Limitations

- Limited dataset scope.
 - Analysis depends on available banking data.
 - Real-time banking integration is not included.
-

Future Scope

- Integration with real-time banking systems.
 - AI-based risk prediction models.
 - Fraud detection using machine learning.
 - Advanced customer segmentation.
 - Predictive financial analytics.
-

Conclusion

The Banking Dashboard & Risk Analytics project successfully demonstrates how data analytics can be used in the banking sector to generate valuable insights and improve decision-making.

Using SQL, Python, and Power BI, the project transformed raw banking data into meaningful visual reports and analytical insights. The project highlights the importance of data visualization, customer analysis, and risk analytics in modern banking systems.

This project also enhanced practical knowledge in data analysis, business intelligence, dashboard development, and banking analytics.

References

1. Power BI Documentation
 2. Python Pandas Documentation
 3. Seaborn & Matplotlib Documentation
 4. PostgreSQL Documentation
 5. Banking Risk Analytics Concepts
-

Project Outcome

This project provides:

- Better understanding of banking data.
- Interactive business intelligence dashboard.
- Risk analytics understanding.
- Practical experience in SQL, Python, and Power BI.
- Strong portfolio project for Data Analyst roles.

